

<https://scam-pied.vercel.app/>

EBS

VOLUME:

1. **a) Create and attach an EBS volume to an EC2 instance, and**
- b) scale the instance by increasing CPU and RAM by changing the instance type.**
- a) Creating and attaching an EBS volume to an EC2 instance**

Step 1: Create a New EBS Volume

Open the EC2 Dashboard.

From the left-hand menu, select Volumes under Elastic Block Store (EBS).

Click Create Volume.

Choose:

Volume type (gp3/gp2)

Size of the volume

Availability Zone (must be the same as the EC2 instance)

Click Create Volume.

Step 2: Attach the EBS Volume to the Existing EC2 Instance

In the Volumes section, select the newly created volume.

Click Actions → Attach Volume.

Select the target EC2 instance from the list.

Specify the device name (example: /dev/xvdf).

Click Attach.

Step 3: Verify the Volume on the EC2 Instance

Connect to the EC2 instance using SSH.

Check the attached disks:

lsblk

Identify the new volume (e.g., /dev/xvdf).

Note: Ensure that the EBS volume and the EC2 instance belong to the **same Availability Zone**, as cross-AZ attachment is not supported.

b) Procedure: increasing/decreasing of CPU and RAM

Step 1. - Navigate to the EC2 dashboard.

Step 2. - Stop the EC2 instance associated with the EBS volume.

Step 3. - Click "Actions" and then—instance settings—change the instance type.
"Modify Volume".

Step 4. - Increase/ decrease the size or change the volume type to a higher performance specification.

Step 5. - Click "Modify" to apply the changes.

Step 6: Now the start the machine you see with increase /decrease of size and performance

2. Attach and permanently mount an EBS volume to a Linux EC2 instance to ensure data persistence across reboots.

Step 1: Create an EBS Volume

1. Log in to **AWS Management Console**
 2. Go to **EC2 → Volumes**
 3. Click **Create Volume**
 4. Choose:
 - **Volume Type:** gp3 (recommended)
 - **Size:** (e.g., 20 GB)
 - **Availability Zone:** Same AZ as EC2 instance
 5. Click **Create Volume**
-

Step 2: Attach the EBS Volume to EC2

1. Select the newly created **EBS volume**
 2. Click **Actions → Attach Volume**
 3. Choose:
 - **Instance:** Select your Linux EC2 instance
 - **Device name:** /dev/xvdf
 4. Click **Attach**
-

Step 3: Connect to the EC2 Instance

Connect using SSH:

```
ssh -i key.pem ec2-user@<public-ip>
```

Step 4: Verify the Attached Disk

Check all available disks:

```
lsblk
```

You will see:

- Root volume: /dev/xvda
- New volume: /dev/xvdf (no partition yet)

Check filesystem:

```
lsblk -fs
```

Step 5: Create a Partition

Create a partition on the new volume:

```
fdisk /dev/xvdf
```

Inside fdisk:

- Press m → Help menu
- Press n → New partition
- Press p → Primary partition
- Press Enter → Default partition number
- Press Enter → Default first sector
- Press Enter → Default last sector (use full disk)
- Press w → Write changes and exit

Notify kernel:

partprobe

Step 6: Verify the New Partition

lsblk -fs

You should now see:

/dev/xvdbf1

Step 7: Format the Partition

Format with XFS filesystem:

mkfs.xfs /dev/xvdbf1

Verify:

lsblk -fs

Step 8: Create a Mount Directory

mkdir /mnt/yogita

Step 9: Mount the Volume

mount /dev/xvdf1 /mnt/yogita

Verify:

df -h

Step 10: Make the Mount Persistent

Get the UUID of the volume:

blkid /dev/xvdf1

Edit fstab:

nano /etc/fstab

Add this line at the end:

Syntax : UUID=<UUID-value> /mnt/yogita xfs defaults,nofail 0 0

example

UUID =f97b4832-90b6-4b31-a18b-34a6a2e39b75 /mnt/yogita xfs defaults,nofail 0 0

Save and exit.

Test fstab:

mount -a

Step 11: Verify Persistence

1. Reboot the instance:

reboot

2. Reconnect and check:

df -h

☐ Volume should be mounted automatically.

->df -h

->lsblk

->lsblk -fs

->sudo fdisk /dev/xvdbf

->Command (m for help): m

->Command (m for help): n

->Partition type

p primary (0 primary, 0 extended, 4 free)

e extended (container for logical partitions)

Select (default p): p

Partition number (1-4, default 1): 1

First sector (2048-8388607, default 2048): 2048

Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-8388607, default 8388607): 8388607

->Command (m for help): w

->lsblk -fs

->sudo mkfs.xfs /dev/xvdbf1

->lsblk -fs

->sudo mkdir /mnt/vol

->sudo mount /dev/xvdbf1 /mnt/vol

```
-> df -h
-> sudo nano /etc/fstab
-> sudo mount -a
-> sudo nano /etc/fstab
-> cd /mnt/vol
-> sudo nano f1.txt
-> cd ..
-> cd ..
-> sudo mount -a
-> df -h
```

Step 12: Detach and Reattach Volume (Data Recovery Scenario)

Detach:

1. EC2 → Volumes → Select volume
2. Actions → Detach volume

Attach to another EC2 instance:

1. Attach to new instance (same AZ)
2. Login to new instance
3. Create mount directory:

```
mkdir /mnt/yogita
```

4. Mount the volume:

```
mount /dev/xvdf1 /mnt/yogita
```

All previous data will be available

Summary Flow

Create Volume → Attach → lsblk → fdisk → mkfs → mkdir → mount → fstab → reboot

3. Create a snapshot of the attached EBS volume and use it to create and attach a new volume to an EC2 instance in another AWS region.

Attaching Volumes across regions of EC2 using the snapshot

To create a snapshot of an EBS volume and attach it to an instance in another region in AWS, you'll need to follow these steps:

Step 1. Create a Snapshot:

- a. Sign in to the AWS Management Console.
- b. Navigate to the EC2 Dashboard.
- c. Click on "Volumes" in the left-hand navigation pane.
- d. Select the EBS volume you want to create a snapshot of.
- e. Click on the "Actions" dropdown menu above the volume list and select "Create Snapshot."
- f. Provide a name and description for the snapshot.
- g. Click on the "Create Snapshot" button to initiate the snapshot creation process.

Step 2. Copy the Snapshot to Another Region:

- a. Once the snapshot is created, go to the "Snapshots" section in the EC2 Dashboard.
- b. Select the snapshot you just created.
- c. Click on the "Actions" dropdown menu above the snapshot list and select "Copy Snapshot."
- d. Choose the destination region where you want to copy the snapshot.
- e. Click on the "Copy Snapshot" button to initiate the copy process. This may take some time depending on the size of the snapshot and the network speed.

Step 3. Monitor the Snapshot Copy Progress:

- a. You can monitor the progress of the snapshot copy by navigating to the "Snapshots" section in the EC2 Dashboard of the source region.
- b. Look for the snapshot you copied and check its status. It will change from "pending" to "completed" once the copy process is finished.

Step 4. Switch to the Destination Region:

- a. Use the region selector in the top-right corner of the AWS Management Console to switch to the destination region where you copied the snapshot.

Step 5. Create a Volume from the Snapshot:

- a. In the EC2 Dashboard of the destination region, go to the "Snapshots" section.
- b. Find the snapshot you copied from the source region.
- c. Click on the snapshot, then click on the "Actions" dropdown menu and select "Create Volume."
- d. Configure the volume settings, such as volume type, size, and availability zone.
- e. Click on the "Create Volume" button to create the volume from the snapshot.

Step 6. Create the ec2 instance & Attach the Volume to an Instance:

- a. Once the volume is created, navigate to the "Volumes" section in the EC2 Dashboard.
- b. Find the newly created volume and select it.
- c. Click on the "Actions" dropdown menu and select "Attach Volume."
- d. the EC2 instance to which you want to attach the volume and specify the device name.
- e. Click on the "Attach" button to attach the volume to the instance.

We have created a snapshot of an EBS volume, copied it to another region, created a volume from the snapshot, and attached it to an instance in the destination region.

EFS:

Step 1: Launching EC2 Instances

1. Sign in to AWS Management Console.
2. Go to the EC2 dashboard.
3. Click on "Launch Instance".
4. Configure instance details:
 - Instance Name: **YogitaINS1**
5. Choose an Amazon Machine Image (AMI) **for Linux**.
6. Select instance type: t2.micro.
 - Key Pair: EFSkey
 - **Network(Edit at Network): Choose Subnet-1a (AZ -1a)**
 - **Security Group: Create a new security group (SG) and add NFS and allow from anywhere.**
7. Launch the instance.
8. Repeat the above steps to launch another instance named YogitaINS2 in **Subnet-1b**(AZ -1b) with the same configuration.

Step 2: Creating an EFS File System

1. Go to the EFS service in the AWS Management Console.
2. Click on "Create file system".
3. Specify details:
 - Name: Optional (Give name YogitaEFSshared)
 - VPC: DefaultClick on Customize
 - Enable region button.Click on "Next".
4. In the Network settings:
 - Delete all security groups. (Except for instance in 1a and 1b)
 - **Select the newly created security group (NFS).**---which is created while creating instanceClick on "Next" " Next"and review the configuration.
Click on "Create file system".

Step 3: Accessing the two EC2 instances named YogitaINS1 & YogitaINS2 in **two different PowerShell** sessions and performing the specified tasks:

Accessing EFS-1 Instances in Two Different PowerShell Sessions:

1. Open Two PowerShell Sessions:
 - Open two separate PowerShell windows or tabs on your local machine.
- For each Instance:

2. SSH into the Instance:

- Use the SSH command to connect to the EFS-1 instance

```
ssh -i [path-to-your-keypair.pem] ec2-user@[instance-public-ip]
```

3. Switch to Root User:

- Gain root access by executing the following command:

```
sudo su
```

4. Create a Directory:

- Make a directory named "efs" using the following command:

```
mkdir efs
```

5. Install Amazon EFS Utilities:

- Use yum package manager to install the Amazon EFS utilities:

```
yum install -y amazon-efs-utils
```

6. List Files:

- Execute the following command to list files in the current directory:

```
ls
```

7. Verify Installation (Optional):

- Optionally, you can verify the installation of the Amazon EFS utilities by checking the version:

```
efs-utils --version
```

Repeat Steps 2-7 for ----- EFS-2 Instance.

Step 4: Attaching EFS to EC2 Instances

1. Go to the EFS service in the AWS Management Console.

2. Click on the target EFS file system.

3. Select and click on EFS YogitaEFSshared

fs

4. Now move down and click on Network to check both Instances have correct/same security groups when they were created.

If not same then click on Manage and select correct one and then save

5. Click on the "Attach" button.

6. Choose "**Mount via DNS**" option

7. Copy the 1st displayed command. And paste on both ec2 instances(powershell) and run.

8. Paste and execute the copied command in the terminal to mount the EFS file system onto the instance.

Step 5: Verify and Test EFS

1. Change the directory to EFS on both instances.

2. **Create a file on one instance. (say f1.txt) touch f1.txt ls**

3. **Verify that the file automatically synchronizes and appears on the other instance.**

Week 5: VPC

Step 1: Create the VPC

1. Go to **AWS Management Console** → **VPC**
2. Click **Create VPC**
3. Choose **VPC only**
4. Enter:
 - **Name:** Custom-VPC
 - **IPv4 CIDR block:** e.g. 10.0.0.0/16
5. Click **Create VPC**

Step 2: Create Subnets (Public & Private)

Create subnets in same **Availability Zones** for high availability.

Example:

- **Public Subnet:** 10.0.1.0/24 (AZ-A)
- **Private Subnet:** 10.0.2.0/24 (AZ-A)

Steps:

1. Go to **Subnets** → **Create subnet**
2. Select your VPC
3. Choose AZ
4. Enter CIDR
5. Create subnet

Step 3: Create an Internet Gateway (IGW)

1. Go to **Internet Gateways**
2. Click **Create internet gateway**
3. Name it Custom-IGW
4. **Attach** it to your VPC

Step 4: Create Route Tables

You need **separate route tables** for public and private subnets.

Public Route Table

1. Go to **Route Tables** → **Create route table**
2. Select VPC
3. Add route:
 - Destination: 0.0.0.0/0
 - Target: **Internet Gateway**
4. Associate with **public subnet**

Private Route Table

- Keep default local route only (no IGW)
- Associate with **private subnet**

Step 5: Enable Auto-Assign Public IP (Public Subnet)

1. Select **Public Subnet**
2. Go to **Edit subnet settings**
3. Enable **Auto-assign public IPv4 address**

Step 6: Configure Security Groups

1. Create a **Security Group**
2. Add inbound rules:
 - HTTP (80) / HTTPS (443)
 - SSH (22) from trusted IP
3. Attach to EC2 instances

Step 7: Launch EC2 Instances

- Public EC2 → Public subnet
- Private EC2 → Private subnet
Attach correct security groups.

Step 8: Test the Setup

- Public EC2 → Internet access
- Private EC2 → Internet via NAT
- Private EC2 → No direct inbound access

WEEK-6 VPC USING BASTION SERVER

1. VPC with Bastion Server

VPC & Subnet Design

VPC CIDR: 10.0.0.0/16

Public Subnet (Bastion Server): 10.0.1.0/24

Private Subnet (Database Server): 10.0.2.0/24

2. Create Internet gateway

Attach YogitaVPC

3. Create Nat gateway

4. Create Route Tables

a. Create RouteTablePublic

b. Attach Public Subnet

c. Edit Route: Attach internet gateway

d. Create RouteTablePrivate

e. Attach Private Subnet

f. Edit Route (Add route: Destination :0.0.0.0/0 Target :Nat gateway)

5. EC2 Instance Creation

Create Database Server Instance - PrivateInstanceDBserver

Subnet: Private Subnet (10.0.2.0/24)

Key pair: dbserver.pem

Operating System: Ubuntu

click on Edit near Network Settings

a) VPC- YogitaVPC

b) Subnet : PrivateSubnetforDBserver

c) Add security rule :

SSH default

Type- MySQL/Aurora Source- My IP

Create Bastion Server Database Server Instance

VPC- YogitaVPC

Operating System: Ubuntu

Key pair: bastion.pem

Subnet: Public Subnet (10.0.1.0/24) –PublicSubnetforBastionserver

Add security rule :

SSH default

Type- MySQL/Aurora Source- Private Instance IP (**PrivateInstanceDBserver**)

6. Open powershell or go to Downloads and open Git Bash here

Copy DB Server Key to Bastion Server

Because: (Database server is in private subnet. Direct SSH from local system is NOT possible. DB server key must be copied to Bastion server)

First copy Public Bastion Instance Public IP address and paste in scp command

Command (PowerShell / Git Bash):

```
scp -i bastion.pem dbserver.pem ubuntu@ <BASTION_PUBLIC_IP>:/home/ubuntu/
```

example: scp -i bastion.pem dbserver.pem ubuntu@3.239.225.180:/home/ubuntu/

File is copied securely using SCP

dbserver.pem will now be available inside Bastion server

7. Connect to Bastion Server

SSH Command:

```
ssh -i bastion.pem ubuntu@3.239.225.180
```

```
eg: ssh -i "bastion.pem" ubuntu@3.239.225.180
```

Above shows ubuntu running on public instance (at 10.0.1.252)

Verify key files:

```
ls
```

Output:

```
dbserver.pem
```

8. Connect from Bastion to Database Server

a. First execute in Git bash command (Important)

chmod 400 dbserver.pem

b. Copy SSH of Dbserver:

```
ssh -i "dbserver.pem" ubuntu@10.0.2.220
```

c. Now SSH it from Bastion ie paste above copied SSH of dbserver into Git bash prompt.

Connection to DB server is successful and ubuntu is now running from dbserver at 10.0.2.220

EX1:

Part (b): EC2 Ubuntu Instance Using PuTTY (.ppk key)

Steps

1. Launch EC2 → Select **Ubuntu**
 2. Create key pair → Select **.ppk**
 3. Save ubuntukeyputty.ppk in **AWSKEYS**
 4. Launch instance
 5. Copy **Public IPv4 Address**
 6. Go to **Security → Security Groups**
 7. Edit inbound rules → Ensure **SSH (Port 22)** is allowed
 8. Save rules
-

PuTTY Configuration

9. Open **PuTTY**
10. Paste **Public IPv4** in *Host Name*
11. Saved Session name: ubuntuputty
12. Go to:

Connection → SSH → Auth → Credentials

13. Click **Browse** → Select .ppk file
14. Click **Open**
15. Accept security alert
16. Login as:

ubuntu

Verification

Run:

whoami

Output:

ubuntu

✓ Connected successfully using PuTTY.

2)

STEP 1: Convert .pem to .ppk

1. Open **PuTTYgen**

2. Click **Load**
3. Select .pem file (change file type → *All Files*)
4. Click **Open**
5. Click **Save private key**
6. Save as convertedpemTOppk.ppk in **AWSKEYS**

✓ Conversion completed

STEP 2: Get EC2 Public IP

AWS Console → EC2 → Instances → **Copy Public IPv4 / DNS**

STEP 3: Configure PuTTY

1. Open PuTTY
2. Host Name: <Public IP / DNS>
3. Go to:

Connection → SSH → Auth → Credentials

4. Browse and select .ppk file
5. Go to:

Connection → Data

6. Enter username:

ubuntu

7. Go back to **Session**
 8. Save session as: pemputtysession
 9. Click **Open**
 10. Accept security prompt
-

STEP 4: Verification

Run:

Whoami **Output:** Ubuntu Type: exit

e. Create Windows EC2 Instance and Connect via RDP

Step 1: Launch a Windows EC2 Instance

Login to your AWS Management Console.

Navigate to EC2 → Instances → Launch Instance.

Select an Amazon Machine Image (AMI).

Choose a Windows Server AMI (e.g., Windows Server 2019 or 2022).

Choose an Instance Type (e.g., t2.micro for free tier or as per your requirement).

Configure Instance Details as needed.

Add Storage if required (default is usually sufficient).

Configure Security Group:

Ensure RDP port 3389 is allowed.

Source can be My IP (for security) or a wider range if needed.

Select or create a new Key Pair.

Download the .pem file (keep it safe; it's needed to decrypt the password).

Click Launch Instance.

Step 2: Get the Public DNS / IP

After the instance is running, go to EC2 → Instances.

Select your instance and note the Public DNS (IPv4) or Public IP. 54.175.28.121

Step 3: Retrieve the Windows Administrator Password

Select your instance → Click Connect → RDP Client.

Click Get Password.

Upload your Key Pair (.pem) file.

Click Decrypt Password.

Copy the Administrator password shown. VyXCbbW1TGJ9=Q8\$e%-OW9CcG7nKOI88

Step 4: Connect via RDP

On your local Windows machine, open Remote Desktop Connection (mstsc).

In the Computer field, enter the Public DNS or IP address of the EC2 instance.

Click Connect.

In the login prompt:

Username: Administrator

Password: Paste the decrypted password from Step 3.

Click OK.

Step 5: You're connected

Your Windows EC2 instance desktop should appear.

You can now use it as a normal Windows machine.